Machine Translated by Google





Internship in System Programming

Attempt 2

Test task description

Chair of Computer Science 11 - RWTH Aachen

April 26, 2025

Commit: d3a2985c

Machine Translated by Google

Table of contents

2 Test task description 2.1 Unit **3**
tests. 3
2.1.1 **os_exec .** 3
2.1.2 **os_initScheduler .** 4
2.2 Error 6
2.3 Critical 8
2.4 Multiple 9
2.5 Summary 9
2.6 Stack Consistency 10
2.7 Scheduling Strategies 11

This document is part of the accompanying materials for the *System Programming internship.* All documents required for this internship are available in the Moodle learning platform. Available for download at https://moodle.rwth-aachen.de . The following email address is available for criticism, suggestions or proposals for improvement:

support.psp@embedded.rwth-aachen.de

Machine Translated by Google

2 Test task description

You can integrate the test tasks by including the respective **progs.c file** in your project. Please note that multiple test tasks cannot be loaded simultaneously, as this would result in multiple programs being defined with the same ID. Also note that in test 2, the idle process should not be running in any test task, as the test tasks are always *ready .* If the idle process is selected in any of the test tasks listed below, the test will be considered failed.

2.1 Unit tests

The tests described below are unit tests. This means they are executed separately from the rest of the operating system. Please note that they are executed before the main function.

2.1.1 os_exec

The unit test **`os_exec`** first checks whether **`os_exec()`** ` returns **`INVALID_PROCESS`** if all slots in the process array are occupied or an invalid program is passed. If **`os_exec`** fails to detect this, a corresponding error message, as shown in Figure 2.1, is displayed. The test then verifies whether eight processes can be created in a completely empty process array and whether the next available slot is used during their initialization.

Next, one of these processes is removed to check if **`os_exec`** can place a process in the resulting gap. Then, the correct initialization of the process array for an instance of program 9 with priority 10 is verified. An incorrectly initialized field in the process array is displayed with a corresponding error message. Finally, it is checked whether the 33 zeros for the runtime context and the high and low bytes of the program address have been correctly written to the process stack. If the registers have been incorrectly initialized or written to the underlying stack (ISR stack), a corresponding error message is issued, as shown in Figure 2.2.

Once all phases have been successfully completed, the LCD displays **"TEST PASSED" .**

Error messages

“Expected invalid processes”
os_exec() did not return **INVALID_PROCESS** even though either **os_processes** is full or an invalid program was passed.

Machine Translated by Google

“Expected valid PID”
os_exec() returned **INVALID_PROCESS ,** although there are still free slots in **os_processes .**

"Not first free slot" os_exec()
did not place a new process in the first free slot of **os_processes .**

“Invalid PID with gap” If only
one slot in the middle of the array is free, **os_exec()** cannot execute a new process and returns **INVALID_PROCESS .**

“Incorrect PID with gap” If only
one slot in the middle of the array is free, **os_exec()** does not use it, overwriting an already occupied slot.

“PID not 0”
The generated process was not assigned to process slot 0.

“Priority not 10” The
idle process does not have priority 10.

"Program pointer invalid" The
function pointer of the idle process does not point to the idle program.

"State not READY" The
idle process has not been set to **OS_PS_READY .**

“SP invalid” The
SP register does not point to the expected memory address.

“Non-zero for register” 33 zeros
were not correctly placed on the process track.

"Invalid high byte" The
high byte for the program address is incorrectly set on the stack.

"Invalid low byte" The
low byte for the program address is incorrectly set on the stack.

"Written into wrong stack" Something
in the stack below was overwritten.

2.1.2 os_initScheduler

The unit test **`os_initScheduler`** checks whether registered programs are correctly initialized **by `os_initScheduler()` .** To do this, in addition to **Program** 0, which is used for the idle task, a dummy program, **Program 1,** is created. Then, **`os_initScheduler`** is called and checks whether the first two slots of the process array have been initialized correctly and the remaining slots are unused. In case of an error, an error message is displayed.

Machine Translated by Google

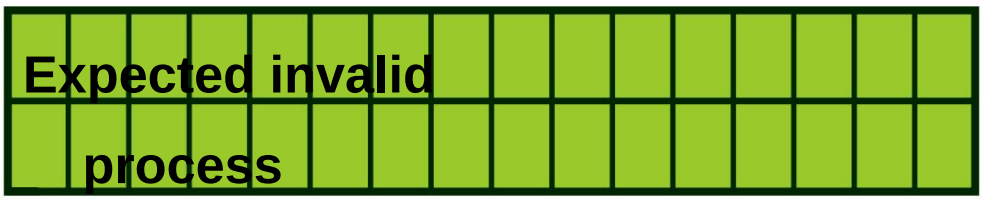


Figure 2.1: **os_exec()** does not return **INVALID_PROCESS**

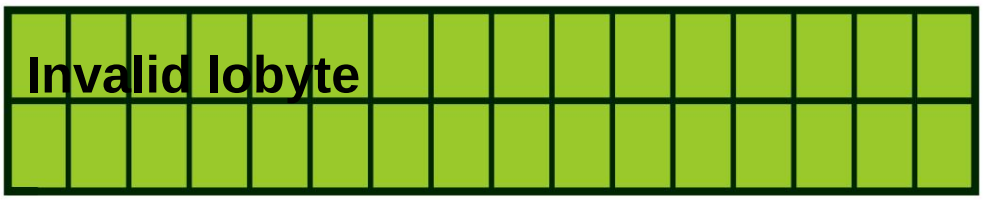


Figure 2.2: Low byte of the program address incorrect

A learning message was issued, as shown in Figure 2.3.
Once all phases have been successfully completed, the LCD displays **"TEST PASSED" .**

Error messages

"Idle not ready"
The idle process is not marked as **OS_PS_READY .**

"Idle not default priority" The
priority of the idle process does not correspond to the **DEFAULT_PRIORITY.**

"Program 1 not started"
Program 1 was not started.

"Prog. 1 not default prio." The
priority of program 1 does not correspond to the **DEFAULT_PRIORITY.**

“Other slots not unused”
Process slots 2 to **MAX_NUMBER_OF_PROCESSES** are not marked as **OS_PS_UNUSED .**

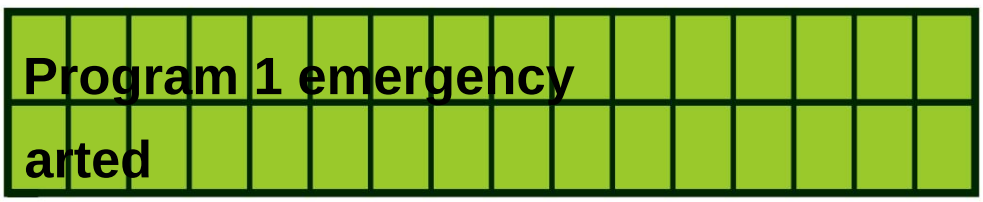


Figure 2.3: Program 1 was not initialized correctly.

Machine Translated by Google

2.2 Error

The *Error* test task checks whether the **`os_errorPstr`** function has been implemented correctly. It goes through five phases. In the first two phases, it checks whether the four buttons on the evaluation board can be used correctly. In the remaining phases, the functionality of ` **os_errorPstr`** is then checked. If you only want to check the functionality of the buttons or only the functionality of **`os_errorPstr` ,** you can configure this using the **`PHASE_BUTTONS`** and **`PHASE_ERROR`** definitions .

Phase 1: In Phase 1, the system checks whether the **DDRC** and **PORTC** registers have been correctly initialized. No external interaction is required during this process. If the initialization is successful, this will be displayed on the LCD.

Phase 2: In phase 2 of the test task, it is tested whether the input of the individual buttons is correctly recognized. For this purpose, the user is prompted to press and release the corresponding button on the evaluation board, as required on the LCD.

Phase 3: In Phase 3, it is checked whether the function **`os_errorPStr`** globally disables interrupts in case of an error. For this, **`os_errorPStr`** is called with the text shown in Figure 2.4. The message must appear on the LCD in this form and be acknowledged by pressing the **Enter** and **ESC** keys . The result of the check is then displayed on the LCD. If an error occurred, the message shown in Figure 2.5 appears on the display.

Phase 4 : In Phase 4, it is checked whether the state of the *Global Interrupt Enable* bit in the status register is correctly restored after an error message is acknowledged, if it was enabled before the call. For this purpose, **`os_errorPstr`** is called with the message **"Confirm error [GIEB on]" .** If the interrupts are incorrectly not reactivated by the implementation of the **`os_errorPStr`** function , the message "Phase 2 of the test failed" appears here (similar to the message shown in 2.5).

Phase 5 : In Phase 5, it is checked whether the state of the *Global Interrupt Enable* bit in the status register is correctly restored after an error message is acknowledged, if it was disabled before the call. For this purpose, **`os_errorPstr`** is called with the message **"Confirm error [GIEB off]" .** If the interrupts are incorrectly enabled by the implementation of the **`os_errorPStr`** function , an error message appears here (similar to the message shown in section 2.5).

Once all phases have been successfully completed, the LCD displays **"TEST PASSED" .**

Machine Translated by Google

Notice

If the test task does not start, but instead the LCD displays *"Booting" ,* there is usually an error in the **os_initScheduler** or the **os_startScheduler .**
Please check these again.

Error messages

“DDR wrong”
The **DDRC** register was not configured correctly for input.

"No pullups" The
pullup resistors were not configured correctly for the buttons.

The ` **waitForInput()**
` function did not wait for a button to be pressed. If this error only appears after a button has been pressed, there may also be a problem in the **`os_getInput()`** function .

"getInput"
The return value of the function **os_getInput()** did not match the expected value.

"waitForNoInput" The
function **os_waitForNoInput()** did not wait until all buttons were released.

“Missing error” The
call or acknowledgment of the function **os_errorPstr** could not be detected.

“Interrupted”
Global interrupts were not disabled in the function **os_errorPstr** when the function was entered.

“GIEB falsely off” After
calling **os_errorPstr ,** global interrupts were not reactivated, even though they were previously enabled.

“GIEB falsely on” After
calling **os_errorPstr ,** global interrupts were enabled even though they were previously disabled.

Machine Translated by Google

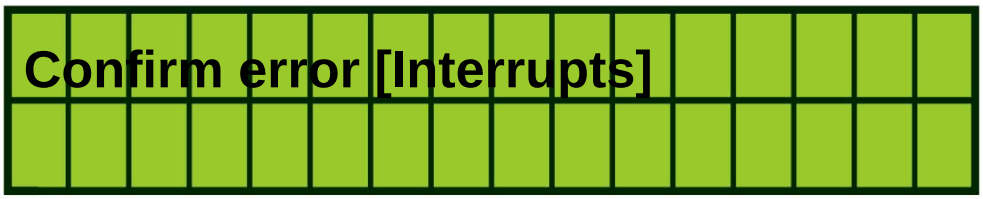


Figure 2.4: Confirm Error

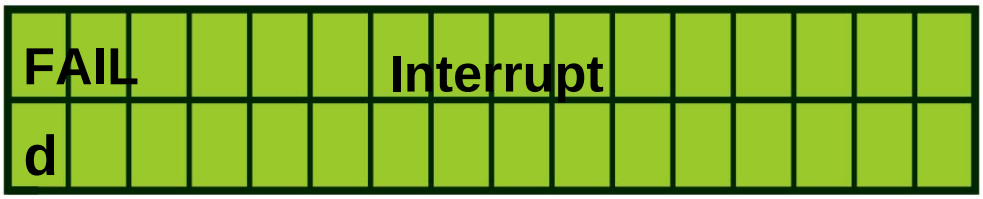


Figure 2.5: Phase 3 failed

2.3 Critical

The *Critical* test task verifies whether entering and exiting critical areas has been correctly implemented. This test task goes through four phases.

Phase 1: In the first phase, it is checked whether entering a critical area only disables the scheduler interrupt. To do this, the "Enter" key on the evaluation board must be pressed within 15 seconds. If the implementation is correct, the phase ends and proceeds to Phase 2. If an error occurs, it is displayed on the LCD and the test task is stopped.

Phase 2: The second phase checks whether counter overflows and underflows are correctly handled with errors. For this, two errors must be acknowledged after the message **"Please confirm Overflow/Underflow"** is displayed. If this is not the case, it will be shown on the LCD.

Phase 3: In the third phase, it is tested whether the GIEB state is correctly restored. For this purpose, a critical section is entered and exited once with the GIEB both switched on and off.

Phase 4 tests whether the critical sections opened in **os_exec()** are closed again in all cases. If so, this is acknowledged on the LCD; otherwise, the error message **"Crit. Section not closed"** appears.

Machine Translated by Google

2.4 Multiple

The *Multiple* test task checks whether an application program can start multiple instances.

A total of six processes are created, five of which execute the same application program. With correct implementation, this will happen continuously.
"1; 2a; 2b; 2c; 2d; 2e;" is displayed on the LCD (see Figure 2.6).
The number describes the application program that is currently running.
An appended letter, on the other hand, reflects an instance of the application program.
again. The order of the strings is arbitrary, however, each one must be repeated.
A possible output may occur at some point. It is also checked whether the condition
The processes are set correctly. If all instances are within 2 minutes
If the test has been successfully started, **"TEST PASSED"** will be displayed.

Error messages

“Current proc not running”
The **OS_PS_RUNNING** state is not set in the process array for the currently running process .

“Other process is running”
Another process besides the currently running one has the state **OS_PS_RUNNING** in
Process array set.

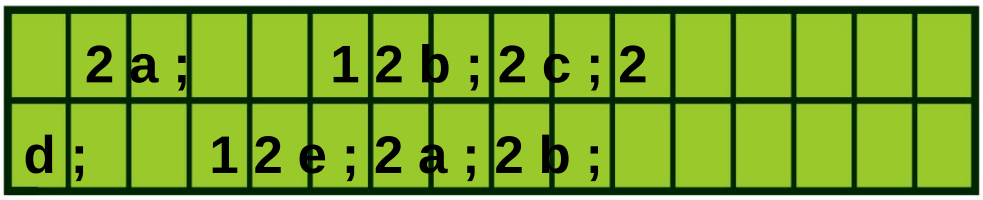


Figure 2.6: Multiple

2.5 Summary

The *Resume* test task checks whether processes are successfully resumed after each scheduler call, i.e., their runtime context must be correctly loaded from the stack and
to be used to continue the program.
For this purpose, one process continuously outputs letters to the LCD, while a second process outputs a digit. However, this digit is only incremented and then repeated.
output after a third process has been successfully invoked. Therefore, with a correct implementation, numbers and letters are output in ascending order according to the

Machine Translated by Google

The image is displayed (see Figure 2.7). If enough numbers and letters are correctly output within 3 minutes, **“TEST PASSED”** is displayed.

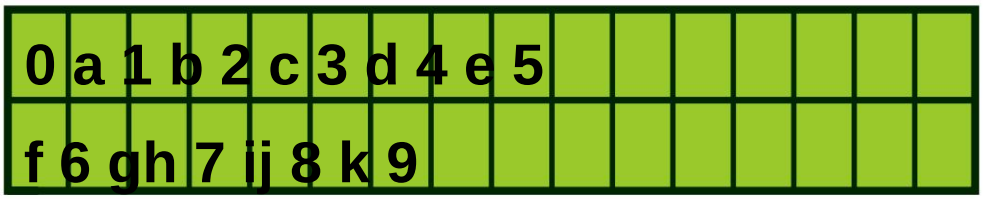


Figure 2.7: Summary

2.6 Stack Consistency

The *Stack Consistency* test task checks whether any inconsistencies in the process stacks are correctly detected. This test task is likely to fail if the *Error* test task has already failed.

For the test, two programs are automatically started, which first check the correctness of their stacked program (SP). Subsequently, the test program generates bit flips in the stack area of the utility program according to predefined patterns and checks whether the scheduler issues a stack inconsistency error message for each flip. This SPOS error message must be acknowledged by simultaneously pressing the **ENTER** and **ESC** keys for the test to continue. The test expects an XOR-based checksum, as described in the documentation, that accurately covers the respective stack. If the stack check behavior does not conform to the specification, the test outputs a non-acknowledgeable error message beginning with **"FAIL ,"** describing the incorrectly handled case (see below).

Once all phases have been successfully completed, the LCD displays **"TEST PASSED" .**

Error messages

"Invalid SP"
The stack pointer does not point to the beginning of the respective process stack.

"Error flagged after init"
An error was detected during initialization.

"Bit below stack detected" A bit
flip was detected below the stack, which is not a stack inconsistency.

"Bit above stack detected" A bit
flip above the stack was detected, which is not a stack inconsistency.

“Double bitflip detected” A
stack inconsistency was detected, but it is not detectable by the expected implementation.

Machine Translated by Google

"Change not detected"
No stack inconsistencies were detected by the operating system.

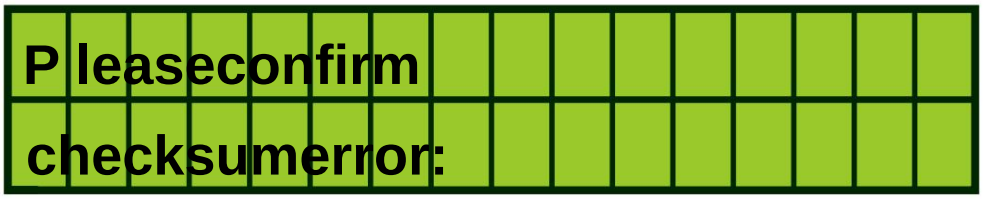


Figure 2.8: Stack Consistency

2.7 Scheduling Strategies

The *Scheduling Strategies* test task can be used to check all predefined scheduling strategies. If only specific strategies are to be tested, the
Others are accordingly deactivated in the defines. Various processes are started for this purpose, and the correct scheduling is checked in four phases:

Phase 1 In the first phase, the process IDs of the processes are listed in order.
The output indicates in which they are allocated processing time by the scheduler. With correct implementation of the scheduling strategies, the following results during the first iteration:
The following output is displayed during the test task phase, followed by an **"OK":**

Even: **“12312312312312312312312312312312”**

Random: (Seed 1) **“13113333113233313212112213111121”**
RoundRobin: **“11222223333333333333333311222223”**

InactiveAging: **“13332333231323332331323332331323”**

RunToCompletion: **“11111111111111111111111111111111”**

If process 0 is selected by a scheduling strategy, the error **"Idle"** appears.
returned". Note that the strategies in the code framework, unless changed,
exhibit this behavior. If an error is detected, the output remains visible and
The first incorrect entry will be marked.

Before calling the random strategy, the internal state of the pseudorandom number generator is reset so that **rand()** returns a known sequence of numbers and
The output of this strategy is therefore also verifiable. For this to be the case, it is important that...
Please follow the implementation instructions.

Phase 2: In Phase 2, it is examined whether the scheduling strategies affect the idle process.
Select this process if no other process can be executed. An error message may appear.

Machine Translated by Google

Issue a message with the appropriate strategy and interrupt further execution.

Phase 3: In the third phase, it is checked whether all processes except the idle process are selected at least once by each strategy. Since the processes do not terminate in this test task, RunToCompletion is excluded from this phase.

Phase 4: This phase tests analogously to the previous one, however, not all process slots are occupied.

Phase 5 This phase checks for all strategies whether the currently running process will be selected again if only this process and the idle process have the **OS_PS_READY** state .

Once all phases have been successfully completed, the LCD displays **"TEST PASSED" .**

Error messages

“Idle returned”
Process 0 was unexpectedly selected.

“S: x **not schedulable”** In strategy
S, process slot x was not selected.

“S: **Idle not scheduled”** In strategy
S, the idle process is not selected.

“S **Expect x, got 0”** Process
x is **READY** and should be selected accordingly, but strategy S returned 0 (idle process) instead.